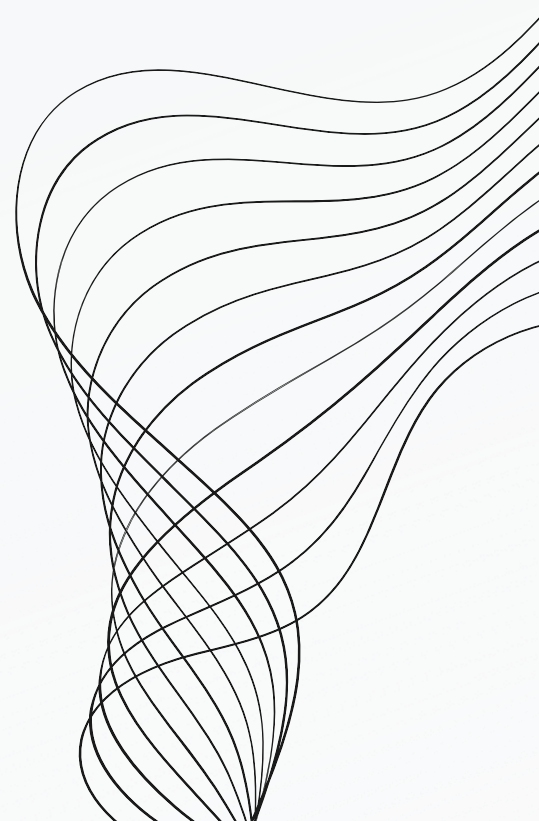
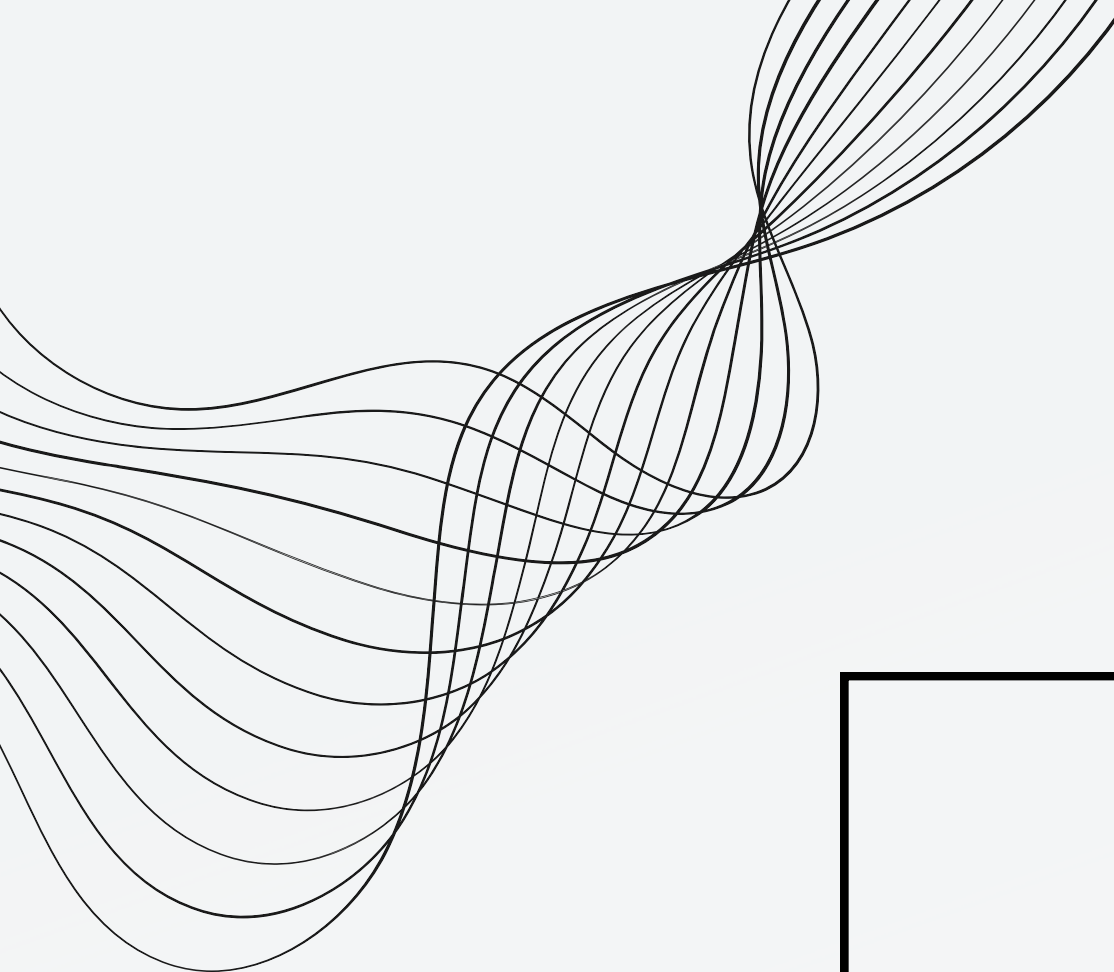




U.T.2

INTRODUCCIÓN A LA PROGRAMACIÓN EN JAVASCRIPT

DESARROLLO WEB EN ENTORNO CLIENTE

CONTENIDO

01

FUNDAMENTOS DE JAVASCRIPT

02

VARIABLES

03

TIPOS DE DATOS

04

OPERADORES

05

ESTRUCTURAS DE CONTROL

06

ERRORES TÍPICOS EN JAVASCRIPT



01

FUNDAMENTOS DE JAVASCRIPT

INTRODUCCIÓN

- JavaScript se diseñó con una sintaxis similar al lenguaje C y aunque adopta nombres y convenciones del lenguaje Java, éste último no tiene relación con JavaScript ya que tienen semánticas y propósitos diferentes.
- JavaScript fue desarrollado originariamente por Brendan Eich, con el nombre de Mocha, el cual se renombró posteriormente a LiveScript y quedó finalmente como JavaScript
- Hoy en día JavaScript es una marca registrada de Oracle Corporation, y es usado con licencia por los productos creados por Netscape Communications y entidades actuales, como la fundación Mozilla.

CARACTERÍSTICAS DE JAVASCRIPT

LIGERO

Ocupa poco espacio en memoria, es fácil de implementar y cuenta con una sintaxis y una semántica simples.

INTERPRETADO

Cada línea de código se lee y se ejecuta de forma secuencial

CASE SENSITIVE

Sensible a mayúsculas y minúsculas

DÉBILMENTE TIPADO

No necesita conocer al dealle con qué tipo de dato se está trabajando. Realiza convesiones de tipos de datos a lo largo de la ejecución del programa

MULTIPARADIGMA

Permite programar aplicaciones completas desde cero aplicando muy distintos paradigmas de programación

MONOHILO

Un único hilo de ejecución se encarga de realizar el trabajo de interpretación del código, se dificultan los interbloqueos y se favorecen las llamadas asíncronas

DINÁMICO

Es capaz de cambiar importantes características del programa mientras se está ejecutando (Ej. Tipo de una variable)

MULTIPLATAFORMA

Permite múltiples interpretes en diferentes navegadores (Google Chorme, Edge, Safari, Opera,...) que se ejecutan en diferentes sistemas operativos (Windows, Mac, Linux,...).

POO

Se modelan patrones a través de clases y se instancian en forma de objetos.

PROGR. IMPERATIVA

Consistente en una secuencia de instrucciones que determinan qué debe hacer la máquina en cada momento. Se centran en el "cómo" de la solución

PROGR. DECLARATIVA

Consiste en describir qué se quiere obtener al finalizar la ejecución del programa. Se centran en el "qué" de la solución

BASADO EN PROTOTIPOS

Los objetos heredan propiedades directamente de otros objetos. Los objetos no se crean instanciando clases, sino clonando o creando objetos

SENTENCIAS

Es una línea de código que realiza una tarea.
En Javascript las sentencias se finalizan con el símbolo de punto y coma. Aunque no es obligatorio ponerlo, pero sí recomendable porque mejora la legibilidad del programa y minimiza errores.

```
abierto = 0;
```

BLOQUES

Es una secuencia de instrucciones encerradas entre llaves

```
if (!cerrado) {  
    abierto = 1;  
    console.log("El candado está abierto");  
}
```

IDENTIFICADORES

Son los nombres que se proporcionan para diferentes estructuras de datos (variables, tipos, funciones,...)

- Identificadores creados por el programador
- Identificadores predefinidos por el lenguaje

Para definir un identificador hay que asegurarse de no violar ninguna de las siguientes reglas:

- Debe empezar obligatoriamente por una letra, guion bajo (_) o el símbolo del dólar (\$).
- Pueden seguirle más letras, números o guiones bajos.
- Distingue entre mayúsculas y minúsculas.
- Puede usar todas las letras que están definidas en UNICODE.

COMENTARIOS

Comentarios de una línea

Se utilizan los caracteres `//`
al principio de la línea

```
// Controla el primer bucle (número de iteraciones)  
var contador = 0;
```

Comentarios de bloque

Se inician con `/*` y finalizan
con `*/`

```
/*  
    Almacena la cantidad de elementos actuales de la estructura.  
    Solo se incluyen los positivos.  
    Debe coincidir con los dispositivos válidos.  
*/
```

ENTRADA - SALIDA BÁSICA

Alert

Es un método que pertenece al objeto window, que veremos más adelante.

El método `window.alert()` (también podemos escribirlo `alert()`) muestra un diálogo de alerta con contenido opcional especificado y un botón OK (Aceptar).

```
alert(message);
```

✓ `message` es un valor opcional del texto que se desea mostrar en el diálogo de alerta.

Confirm

El método `confirm()` muestra una ventana de diálogo con un mensaje opcional y dos botones, Aceptar y Cancelar.

```
let result = confirm(message);
```

✓ `message` es la cadena que se muestra opcionalmente en el diálogo.

✓ `result` es un valor booleano indicando si se ha pulsado Aceptar o Cancelar (Aceptar devuelve `true` y Cancelar devuelve `false`).

ENTRADA - SALIDA BÁSICA

Prompt

El método `prompt()` muestra un diálogo con mensaje opcional, que solicita al usuario que introduzca un texto.

```
let result = prompt(message, default);
```

- ✓ `result` es una cadena de texto que contiene el valor introducido por el usuario, o `null`.
- ✓ `message` es una cadena de texto que se mostrará al usuario. Este parámetro es opcional y puede ser omitido si no se necesita mostrar nada en la ventana.
- ✓ `default` es una cadena de texto que contiene el valor predeterminado para el texto de entrada.

El valor que devuelve es un valor de tipo cadena, si lo que necesitamos es trabajar con valores numéricos debemos realizar el cambio de tipo cadena al tipo numérico con el que necesitamos trabajar, para ello tenemos dos funciones `parseInt` y `parseFloat`, que realizan la conversión a un valor entero y a un valor real.

ENTRADA - SALIDA BÁSICA

document.write

Nos va a permitir insertar un contenido que puede ser texto o un elemento HTML dentro de nuestro código HTML.

Podemos mostrar una cadena de caracteres, el contenido de una variable, el resultado de una expresión, el resultado de la ejecución de una función....

La sintaxis es la siguiente:

```
document.write("Hola qué tal??");
```

En el caso de que le estemos pasando una cadena que contenga código HTML, lo va a interpretar y nos va a mostrar la acción que realice el código HTML, en lugar, de la cadena del código.

```
document.write("<h1> hola qué tal??<h1>");
```

En este caso se mostrará por pantalla un encabezado de tipo h1 con el texto hola qué tal??



Hoy en día `document.write()`, está en desuso no es necesaria ya que se puede reemplazar por otras opciones como `textContent` o `innerHTML`, que son más eficientes y rápidas.

LA CONSOLA DE JAVASCRIPT

- En la consola de Javascript se pueden escribir funciones o sentencias de Javascript que estarán actuando en la página que se encuentra en la pestaña actual del navegador. De esta forma podremos observar los resultados que nos devuelve en la consola al realizar diferentes acciones.

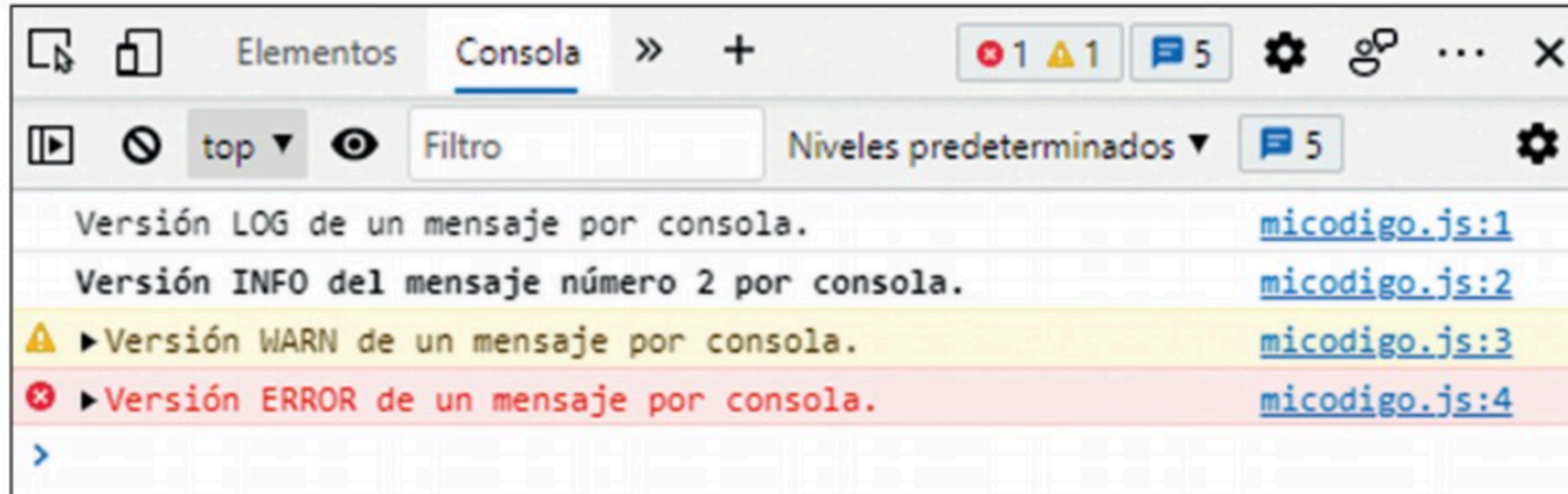
Para acceder a la Consola:

- CTRL+SHIFT+I y Pestaña Console
- CTRL+SHIFT+J
- F12

Función	Descripción
<code>console.log()</code>	Muestra la información proporcionada en la consola Javascript.
<code>console.info()</code>	Equivalente al anterior. Se utiliza para mensajes de información.
<code>console.warn()</code>	Muestra información de advertencia. Aparece en amarillo.
<code>console.error()</code>	Muestra información de error. Aparece en rojo.
<code>console.clear()</code>	Limpia la consola. Equivalente a pulsar CTRL + L o escribir <code>clear()</code> .

MENSAJES GENERADOS POR LA CONSOLA

```
console.log("Versión LOG de un mensaje por consola.");  
console.info("%cVersión INFO del mensaje número %d por consola.,"font-weight: bold;",2);  
console.warn("Versión WARN de un mensaje por consola.");  
console.error("Versión ERROR de un mensaje por consola.");
```





02

VARIABLES

VARIABLES



Una variable es una zona de memoria a la que se asigna un nombre y en la que se guarda un valor. El tipo de dato que va a guardar la variable dependerá del valor que le asignemos. Para utilizar una variable, hay que declararla, en JavaScript podemos utilizar una variable sin haberla declarado, pero esto es algo que debemos evitar.

Para declarar una variable vamos a utilizar la palabra reservada `var` o `let` seguida de un identificador, que es muy importante que sea significativo.

```
let edad;
```

Otra forma de declarar una variable es darle un valor en la declaración (inicialización)

```
let edad = 38;
```

JavaScript **no es un lenguaje tipado** por lo que no tendremos que indicar el tipo de información que vamos a guardar en una variable, es más, vamos a poder guardar informaciones de diferentes tipos en la misma variable.

```
let valor = 38;
```

```
valor = "Hola";
```

```
valor = 3.58;
```

Esto que acabamos de hacer es válido, aunque no aconsejable.

VARIABLES



A la hora de dar nombres a los identificadores de las variables, tendremos que utilizar nombres que realmente describan el contenido de la variable. No podremos usar palabras reservadas, ni símbolos de puntuación en el medio de la variable, ni espacios en blanco. Los nombres de las variables han de construirse con caracteres alfanuméricos y el carácter subrayado (_).

Hay 3 formas de declarar variables en JS:

- **var:** Declara una variable con ámbito global
- **let:** Declara una variable con ámbito local, variable que pierde su valor fuera del bloque, donde está siendo utilizada.
- **const:** Define constantes, variables a las que se asigna un valor que no cambiará nunca.



- **Variable global:** variable definida fuera de la función. Resultan visibles y disponibles para todas las sentencias de un script.
- **Variable local:** es aquella cuyo ámbito se restringe a la función donde se declaro. Sólo resultan visibles y disponibles dentro de la función en la que están definidas.

LET

La palabra reservada `let` va a ser utilizada para declarar variables bloque, el alcance (ámbito o scope) es el bloque donde ha sido declarada. Si intentamos utilizar una variable declarada con `let` más allá del bloque en el cual se encuentra recibiremos en la pantalla un mensaje de error.

Las variables declaradas con `let`, además, no se pueden volver a declarar en el interior del mismo bloque, sino solamente en bloques diferentes y por tanto son variables diferentes.

```
let nombre = "Pepito";
```

VAR

La palabra reservada `var` no está limitada al bloque en el que ha sido declarada, se utiliza para declarar variables globales, en este caso el alcance es todo el programa, por lo que su utilización no es recomendada.

En este caso, la variable puede ser utilizada más allá del bloque en el cual ha sido declarada y, a diferencia de la variable `let`, la variable `var` puede ser de nuevo declarada en el interior del mismo bloque. En este caso el primer valor será reemplazado.

```
var edad = 25;
```

LET VS VAR

- ✓ El ámbito: Mientras que `let` crea una variable cuyo alcance es de bloque, la palabra clave `var` crea una variable global, por lo que es preferible crear variables con `let`.
- ✓ La re-declaración de una variable: No es posible utilizar en el interior del mismo bloque `let` para crear una variable con un identificador que hayamos utilizado anteriormente en el bloque. Con la palabra clave `var`, por el contrario, se es posible re-declarar las variables sin problemas.
- ✓ Hoisting: Las variables `let` no se pueden utilizar antes de la línea en la cual son declaradas, mientras que las variables `var` son inicializadas ya desde el inicio del bloque en el que aparecen, por lo que podríamos utilizar una variable sin haberla declarado anteriormente, esto debemos evitarlo.
- ✓ Propiedades globales: Si las variables `var` se utilizan más allá de una función crea una variable global accesible a través del objeto `window`, que no sucede por ejemplo con la variable `let`.

CONST



Cuando vamos a utilizar una variable que no va a cambiar de valor el ECMAScript 6 (ES6), recomienda la utilización de constantes. Es importante tener en cuenta que no vamos a poder cambiar su valor y que tenemos que iniciar la constante en la declaración.

```
const nombre = "pepito" // En este caso la cte nombre no va a cambiar durante el programa.
```

Una constante es un tipo inmutable, no puede cambiar una vez definida, al igual que let su ámbito es de bloque. Son referencias inmutables, pero sus valores no necesariamente, por ejemplo, en los objetos y arreglos se pueden agregar, modificar o eliminar elementos y propiedades de los mismos, siempre y cuando el objeto o arreglo no cambie de tipo de dato.



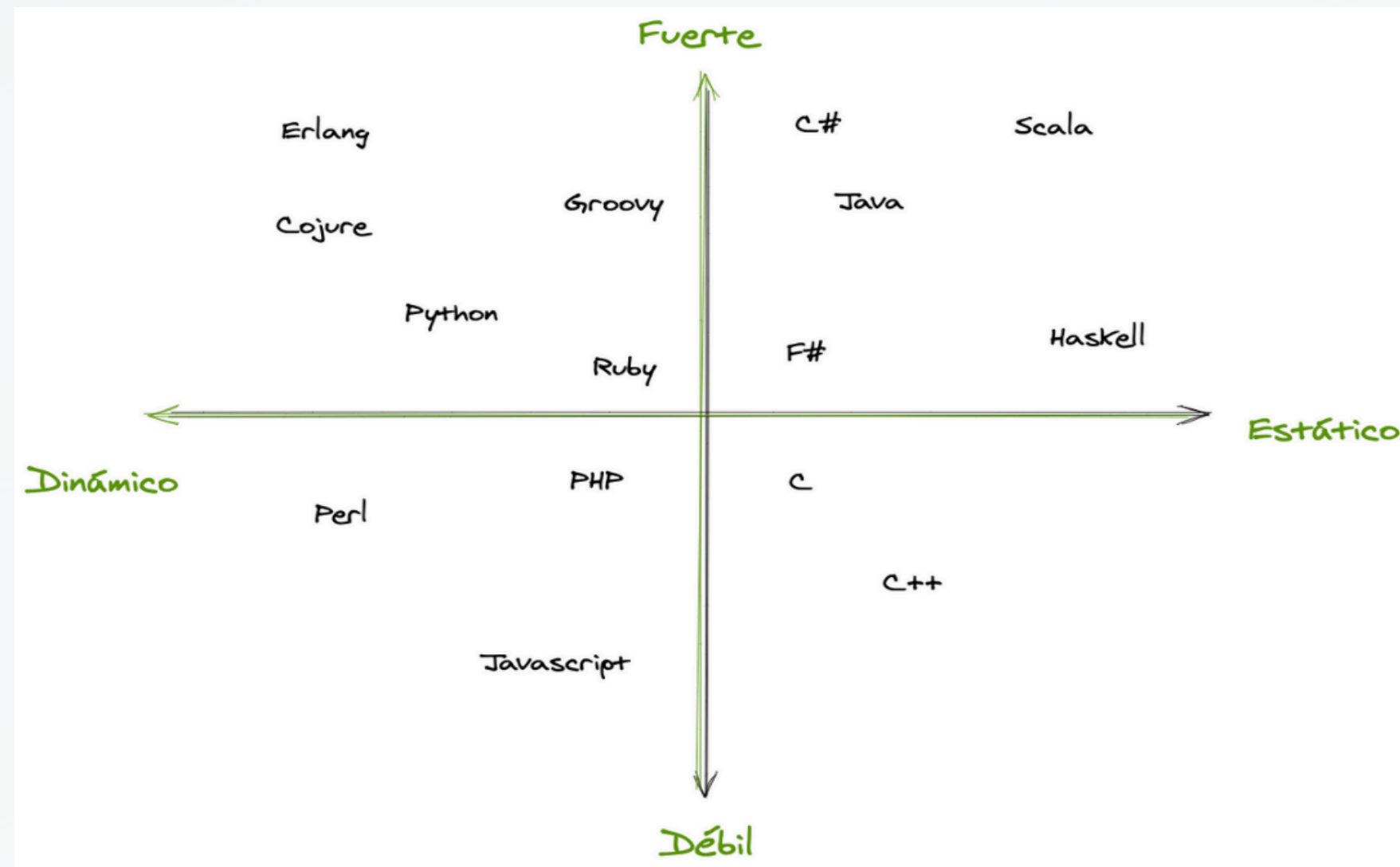
03

TIPOS DE DATOS

DATOS EN JAVASCRIPT

JavaScript es un lenguaje de tipado débil. Los datos en JavaScript no están asociados con ningún tipo de dato.

JavaScript es un lenguaje dinámico, esto significa que las variables toman el tipo en función de la información que almacenan, de tal forma, que podemos dar un valor a una variable con una cadena de texto y luego asignarle un valor numérico



TIPOS DE DATOS EN JAVASCRIPT

PRIMITIVOS

Se accede directamente al valor

COMPLEJOS

Se accede a la referencia del valor

Tipo de dato	Descripción	Ejemplo básico
NUMBER Number	Valor numérico (enteros, decimales, etc...)	42
BIGINT BigInt	Valor numérico grande	1234567890123456789n
STRING String	Valor de texto (cadenas de texto, caracteres, etc...)	'MZ'
BOOLEAN Boolean	Valor booleano (valores verdadero o falso)	true
UNDEFINED undefined	Valor sin definir (variable sin inicializar)	undefined
FUNCTION Function	Función (función guardada en una variable)	function() {}
SYMBOL Symbol	Símbolo (valor único)	Symbol(1)
OBJECT Object	Objeto (estructura más compleja)	{}

TIPOS PRIMITIVOS

Son simples e inmutables, lo que significa que su valor no puede cambiar una vez creado. Se almacenan por valor en la memoria.

Tipo	Descripción	Ejemplo
String	Texto	let saludo = "Hola";
Number	Números (enteros y decimales)	let edad = 25;
BigInt	Números enteros grandes	let big = 12345678901234567890n;
Boolean	true o false	let esMayor = true;
Undefined	Variable sin valor asignado	let x;
Null	Ausencia intencional de valor	let dato = null;
Symbol	Identificadores únicos	let id = Symbol("id");

Características

- ✓Inmutables (no se pueden modificar directamente).
- ✓Se almacenan en la pila (stack).
- ✓Comparación por valor (=== compara directamente los valores).
- ✓Ocupan menos memoria.

TIPOS COMPLEJOS

Son estructuras dinámicas y mutables, que pueden contener múltiples valores. Se almacenan por referencia.

Tipo	Descripción	Ejemplo
Object	Conjunto de pares clave-valor	<code>let persona = { nombre: "Ana", edad: 30 };</code>
Array	Lista ordenada de valores	<code>let numeros = [1, 2, 3];</code>
Function	Bloque de código reutilizable	<code>function saludar() { return "Hola"; }</code>
Date	Fechas y horas	<code>let fecha = new Date();</code>

Características:

- ✓Mutables
- ✓Se almacenan en el heap del navegador.
- ✓Comparación por referencia

String

Un String es una secuencia de caracteres que se utiliza para representar texto

Representar cadenas

```
miString_1 = "con comillas dobles";  
miString_2 = 'con comillas simples';  
miString_3 = `con comillas invertidas o backticks`;
```

Operador de concatenación: +

```
subcadena_1 = "Primera parte, ";  
subcadena_2 = "parte concatenada";  
subcadena_3 = ". Última parte añadida";  
cadenaFinal = "Cadena: " + subcadena_1 + subcadena_2 + subcadena_3 + ".";
```

`${}`

Los backticks permiten incluir expresiones dentro de las llaves de `${}`

```
cadenaFinal = `Cadena: ${subcadena_1}${subcadena_2}${subcadena_3}.`;
```

```
base = 3;  
altura = 22;  
document.write(`El área de nuestro triángulo es: ${base*altura/2}`);
```

Lista de las secuencias de escape más utilizadas

Secuencia	
<code>\'</code>	Comillas simples
<code>\"</code>	Comillas dobles
<code>\\</code>	Barra invertida o <i>backslash</i>
<code>\b</code>	Retroceso
<code>\f</code>	Salto de página
<code>\n</code>	Salto de línea
<code>\r</code>	Retorno de carro
<code>\t</code>	Tabulador horizontal
<code>\v</code>	Tabulador vertical



Utilizando el patrón `\u{código}`, siendo código un valor hexadecimal de hasta 8 dígitos, se puede representar cualquier símbolo de la tabla UNICODE. (Ej.: `\u{20BF}`)

Number

En Javascript Number hace referencia a un único tipo de dato numérico que recoge cualquier formato de número con o sin decimales

Expresar números

```
const num = 4
```

```
const decimal = 15.8;
```

```
const exp = 2e9;
```

Sistema de representación

Se escribe el dígito cero (0) seguido de un carácter de identificación del sistema digital. Binario (b), Octal (o) y Hexadecimal (x)

```
let numeroBinario = 0b10100
```

```
let numeroOctal = 0o45
```

```
let numeroHexadecimal = 0x5A
```

NaN

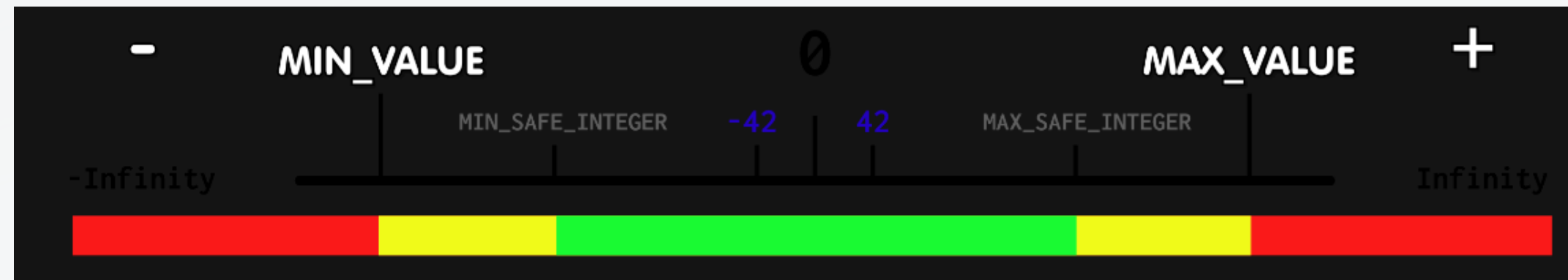
El acrónimo NaN es un valor especial de Javascript que significa literalmente Not A Number (No es un número). El valor NaN se usa para representar valores numéricos que son indeterminados o imposibles de representar como número. Ej: división entre 0

Infinity

La propiedad global Infinity es un valor numérico que representa el infinito. El valor Infinity (infinito positivo) es mayor que cualquier otro número.

BigInt

Con BigInt se puede almacenar y operar de forma segura con números enteros muy grandes, más allá del límite seguro de los enteros number.
Para utilizar el tipo de dato BigInt simplemente se añade una n al final del número



Boolean

Un booleano es un dato lógico que solo puede tomar dos valores: true o false

Representar booleanos

```
valorVerdadero = true;  
valorFalso = false;
```



A la hora de evaluar expresiones lógicas se considera que todo lo que sea igual a cero es falso y todo lo que sea distinto de cero es verdadero

undefined

En Javascript se obtiene el valor **undefined** cuando todavía no se le ha asignado un valor a una variable y, en general, cuando no es capaz de evaluar una expresión.

```
Boolean(undefined) = false
```

null

En Javascript el valor null representa el valor vacío o nulo.

```
Boolean(null) = false
```


¿Qué tipo de dato tiene?

Hay varias formas de saber que tipo de dato tiene una variable en Javascript:

typeof()

Podemos utilizar la función `typeof`, que nos devuelve el tipo de dato de la variable que le pasemos por parámetro.

```
console.log(typeof text);    // "String"  
console.log(typeof number); // "Number"  
console.log(typeof boolean); // "Boolean"  
console.log(typeof notDefined); // undefined
```

constructor.name()

Más adelante, nos encontraremos que en muchos casos, `typeof()` resulta insuficiente porque en tipos de datos más avanzados simplemente nos indica que son objetos. Con `constructor.name` podemos obtener el tipo de constructor que se utiliza, un concepto que veremos más adelante.

Conversión entre tipos de datos

Aunque los datos en JavaScript son muy sencillos, por el hecho de ser dinámicos, se producen conversiones automáticas al realizar operaciones.

Por ejemplo, cuando intentamos sumar dos números:

```
4 + 5 // resultado = 9
```

Si uno de esos números está en formato de cadena de texto, JavaScript lo que hará es intentar convertir el otro número a una cadena y los concatenará, por ejemplo:

```
4 + "5" // resultado = "45"
```

Otro ejemplo podría ser:

```
4 + 5 + "6" // resultado = "96"
```

Esto puede resultar ilógico, pero sí que tiene su lógica. La expresión se evalúa de izquierda a derecha. La primera operación funciona correctamente devolviendo el valor de 9 pero al intentar sumarle una cadena de texto "6" JavaScript lo que hace es convertir ese número a una cadena de texto y se lo concatenará al comienzo del "6".

Conversión entre tipos de datos

Si lo que deseas es realizar la conversión de números a cadenas, es mucho más sencillo, ya que simplemente tendrás que concatenar una cadena vacía al principio, y de esta forma el número será convertido a su cadena equivalente:

```
("" + 3400) // resultado = "3400"  
("" + 3400).length // resultado = 4
```

En el segundo ejemplo podemos ver la gran potencia de la evaluación de expresiones. Los paréntesis fuerzan la conversión del número a una cadena. Una cadena de texto en JavaScript tiene una propiedad asociada con ella que es la longitud (`length`), la cual te devolverá en este caso el número 4, indicando que hay 4 caracteres en esa cadena "3400". La longitud de una cadena es un número, no una cadena.

Para convertir cadenas a números dispones de las funciones: `parseInt()` y `parseFloat()`.

parseInt

Conversión de Cadenas a Números

parseInt es una función de alto nivel que nos permite hacer la conversión de valor de tipo cadena de caracteres a un valor numérico de tipo entero y que no está asociada a ningún objeto.

```
let valor_entero = parseInt(cadena);
```

La función parseInt comprueba el primer argumento, una cadena, e intenta devolver un entero.

Por ejemplo:

```
parseInt("34") // resultado = 34  
parseInt("89.76") // resultado = 89  
4 + 5 + parseInt("6") // resultado = 15
```

parseFloat

Conversión de Cadenas a Números

parseFloat es una función de alto nivel, que nos permite hacer la conversión de un valor de tipo cadena de caracteres a un valor numérico de tipo real (float) y que no está asociada a ningún objeto.

```
let valor_real = parseFloat(cadena);
```

Si el primer carácter no se puede convertir a número, parseFloat devuelve NaN. Para fines aritméticos, el valor NaN no es un número para ninguna base. Puede llamar a la función isNaN para determinar si el resultado de parseFloat es NaN. Si se pasa NaN en operaciones aritméticas, la operación resultante también será NaN.

parseFloat devolverá un entero o un número real según el caso:

```
parseFloat("34") // resultado = 34
```

```
parseFloat("89.76") // resultado = 89.76
```



04

OPERADORES

OPERADORES DE COMPARACIÓN

Los operadores de comparación son aquellos que utilizamos en nuestro código para realizar comprobaciones. Estas expresiones de comparación devuelven un booleano con un valor de true o false.

Nombre	Operador	Descripción
Operador de igualdad <code>=</code>	<code>a == b</code>	Comprueba si el valor de <code>a</code> es igual al de <code>b</code> . No comprueba tipo de dato.
Operador de desigualdad <code>!=</code>	<code>a != b</code>	Comprueba si el valor de <code>a</code> no es igual al de <code>b</code> . No comprueba tipo de dato.
Operador mayor que <code>></code>	<code>a > b</code>	Comprueba si el valor de <code>a</code> es mayor que el de <code>b</code> .
Operador mayor/igual que <code>>=</code>	<code>a >= b</code>	Comprueba si el valor de <code>a</code> es mayor o igual que el de <code>b</code> .
Operador menor que <code><</code>	<code>a < b</code>	Comprueba si el valor de <code>a</code> es menor que el de <code>b</code> .
Operador menor/igual que <code><=</code>	<code>a <= b</code>	Comprueba si el valor de <code>a</code> es menor o igual que el de <code>b</code> .
Operador de identidad <code>===</code>	<code>a === b</code>	Comprueba si el valor y el tipo de dato de <code>a</code> es igual al de <code>b</code> .
Operador no idéntico <code>!==</code>	<code>a !== b</code>	Comprueba si el valor y el tipo de dato de <code>a</code> no es igual al de <code>b</code> .

OPERADORES ARITMÉTICOS

Para realizar operaciones matemáticas básicas

Nombre	Operador	Descripción
Suma	<code>a + b</code>	Suma el valor de <code>a</code> al valor de <code>b</code> .
Resta	<code>a - b</code>	Resta el valor de <code>b</code> al valor de <code>a</code> .
Multiplicación	<code>a * b</code>	Multiplica el valor de <code>a</code> por el valor de <code>b</code> .
División	<code>a / b</code>	Divide el valor de <code>a</code> entre el valor de <code>b</code> .
Módulo	<code>a % b</code>	Devuelve el resto de la división de <code>a</code> entre <code>b</code> .
Exponenciación	<code>a ** b</code>	Eleva <code>a</code> a la potencia de <code>b</code> , es decir, <code>a^b</code> . Equivalente a <code>Math.pow(a, b)</code> .

OPERADORES DE ASIGNACIÓN

Estos operadores nos permiten asignar información a diferentes constantes o variables a través del símbolo =

Nombre	Operador	Descripción
Asignación	<code>c = a + b</code>	Asigna el valor de la parte derecha
Suma y asignación	<code>a += b</code>	Es equivalente a <code>a = a + b.</code>
Resta y asignación	<code>a -= b</code>	Es equivalente a <code>a = a - b.</code>
Multiplicación y asignación	<code>a *= b</code>	Es equivalente a <code>a = a * b.</code>
División y asignación	<code>a /= b</code>	Es equivalente a <code>a = a / b.</code>
Módulo y asignación	<code>a %= b</code>	Es equivalente a <code>a = a % b.</code>
Exponenciación y asignación	<code>a **= b</code>	Es equivalente a <code>a = a ** b.</code>

OPERADORES UNARIOS

Los operadores unarios son aquellos que en lugar de tener dos operandos, como los anteriores, sólo tienen uno. Es decir, se realizan sobre un sólo valor almacenado en una variable.

Nombre	Operador	Descripción
Incremento	<code>a++</code>	Usa el valor de <code>a</code> y luego lo incrementa. También llamado postincremento .
Decremento	<code>a--</code>	Usa el valor de <code>a</code> y luego lo decrementa. También llamado postdecremento .
Incremento previo	<code>++a</code>	Incrementa el valor de <code>a</code> y luego lo usa. También llamado preincremento .
Decremento previo	<code>--a</code>	Decrementa el valor de <code>a</code> y luego lo usa. También llamado predecremento .
Resta unaria	<code>-a</code>	Cambia de signo (niega) a <code>a</code> .

OPERADORES BINARIOS

Existen los denominados operadores a nivel de bit. Se trata de una serie de operadores que nos permiten realizar operaciones básicas trabajando a nivel binario, donde los operandos solo pueden tomar valores de 0 o 1:

Nombre	Operador	Descripción
Operador AND	<code>a & b</code>	Devuelve 1 si ambos operandos son 1 .
Operador OR	<code>a b</code>	Devuelve 1 si al menos un operando es 1 .
Operador XOR (OR exclusivo)	<code>a ^ b</code>	Devuelve 1 si ambos operandos son diferentes.
Operador NOT (unario)	<code>~a</code>	Invierte los bits del operando (por ejemplo, 000101 pasa a 111010). Trunca a 32 bits.
Operador LEFT SHIFT	<code>a << b</code>	Desplazamiento de bits hacia la izquierda. Ej: 11 (3) pasa a 110 (6).
Operador RIGHT SHIFT	<code>a >> b</code>	Desplazamiento de bits hacia la derecha. Ej: 11 (3) pasa a 1 (1).
Operador RIGHT SHIFT sin signo	<code>a >>> b</code>	Desplazamiento de bits hacia la derecha, como un operador sin signo.

a	b	AND	OR	XOR	NOT AND	NOT OR	NOT X
0	0	0	0	0	1	1	1
0	1	0	1	1	1	0	0
1	0	0	1	1	1	0	0
1	1	1	1	0	0	0	1

OPERADORES LÓGICOS

Los operadores lógicos son uno de los tipos de operadores más importantes de cualquier lenguaje de programación puesto que son la forma de evaluar condiciones que permitirán tomar decisiones a lo largo del flujo de ejecución del programa

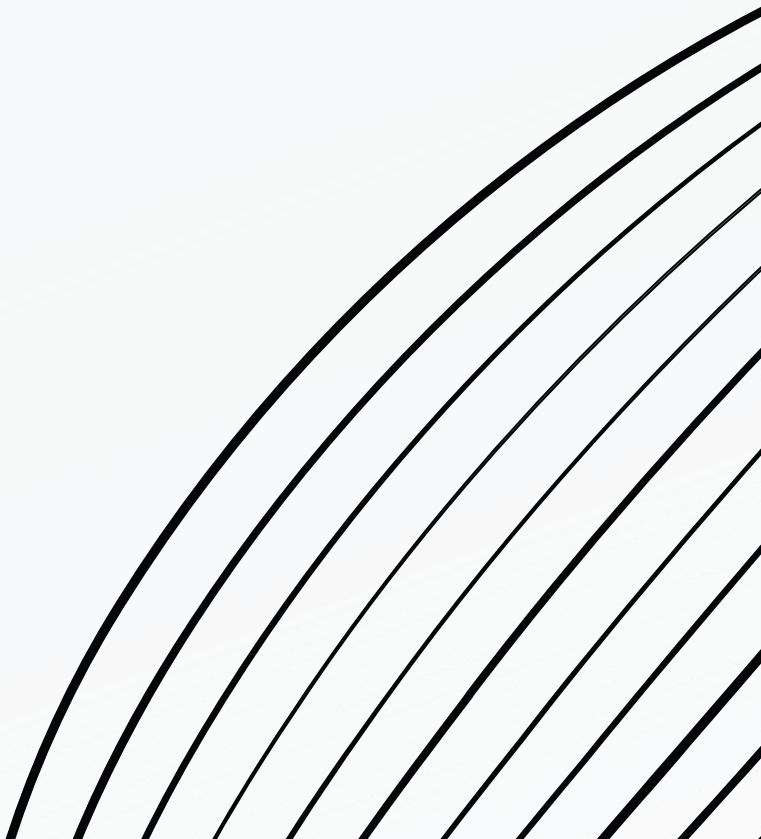
Nombre	Operador	Descripción
Operador lógico AND	<code>a && b</code>	Devuelve <code>a</code> si es <code>false</code> , sino devuelve <code>b</code> .
Operador unario lógico NOT	<code>!a</code>	Invierte el valor. Si es <code>true</code> devuelve <code>false</code> y viceversa.
Operador lógico OR	<code>a b</code>	Devuelve <code>a</code> si es <code>true</code> , sino devuelve <code>b</code> .
Operador ternario ?:	<code>a ? b : c</code>	Si <code>a</code> es <code>true</code> , devuelve <code>b</code> , sino devuelve <code>c</code> .

Tablas de verdad en operadores lógicos

a	b	a && b
0	0	0
0	1	0
1	0	0
1	1	1

a	b	a b
0	0	0
0	1	1
1	0	1
1	1	1

a	!a
0	1
1	0



OPERADORES DE CADENA

El operador de cadena `+` se utiliza para concatenar cadenas de caracteres.

Nombre	Operador	Descripción
Concatenación de texto	<code>a + b</code>	Une el contenido de <code>a</code> con el contenido de <code>b</code>



05

ESTRUCTURAS DE CONTROL

ESTRUCTURAS DE CONTROL

Las estructuras de control son instrucciones que permiten alterar el flujo de ejecución de un programa, de manera que habrá líneas de código que se ejecuten y otras que no. El elemento que determinará esas trayectorias es la condición, normalmente construida con expresiones lógicas.

SELECTIVAS

REPETITIVAS

DE SALTO

ESTRUCTURAS SELECTIVAS

if

```
if (condición) {  
    instrucciones_si_true;  
}  
else {  
    instrucciones_si_false;  
}
```

Ejemplo:

```
if (miEdad > 30){  
    alert("Ya eres una persona adulta");  
}
```


ESTRUCTURAS SELECTIVAS

if ... else if ... else

```
if (condición1) {  
    instrucciones_si_condición1_true;  
}  
else if (condición2) {  
    instrucciones_si_condición2_true;  
}  
...  
else if (condiciónN) {  
    instrucciones_si_condiciónN_true;  
}  
...  
else {  
    instrucciones_si_todas_condiciones_false;  
}
```

Ejemplo:

```
if (miEdad > 30) {  
    alert("Ya eres una persona adulta.");  
} else {  
    alert("Eres una persona joven.");  
}
```

ESTRUCTURAS SELECTIVAS

switch

```
switch (expresión) {  
    case valor_1:  
        instrucciones_1  
        [break;]  
    case valor_2:  
        instrucciones_2  
        [break;]  
    ...  
    default:  
        instrucciones_predeterminadas  
        [break;]  
}
```

```
switch (new Date().getDay()) {  
    case 0:  
        dia = "Domingo";  
        break;  
    case 1:  
        dia = "Lunes";  
        break;  
    case 2:  
        dia = "Martes";  
        break;  
  
    case 3:  
        dia = "Miércoles";  
        break;  
    case 4:  
        dia = "Jueves";  
        break;  
    case 5:  
        dia = "Viernes";  
        break;  
    case 6:  
        dia = "Sábado";  
    default:  
        dia = "Número de día incorrecto";  
}
```

ESTRUCTURAS REPETITIVAS

for

```
for ([expresiónInicial]; [expresiónCondicional]; [expresiónDeActualización]) {  
    instrucciones;  
}
```

Ejemplo:

```
for (let i=1; i<=20; i++) {  
    // instrucciones que se ejecutarán 20 veces.  
}
```

ESTRUCTURAS REPETITIVAS

for

```
for (variable in objeto)  
  instrucción
```

```
const dias = ["Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo"];  
for(let i in dias) {  
  alert(dias[i]);  
}
```



ESTRUCTURAS REPETITIVAS

for

```
for (variable of iterable) {  
  statement  
}
```

Es un bucle parecido al anterior, la diferencia está en que en lugar de devolvernos el índice del elemento de la estructura iterable que estamos recorriendo, en este caso nos devuelve el elemento.

```
const dias = ["Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo"];  
  
for(let dia of dias) {  
  console.log(dia);  
}
```


ESTRUCTURAS REPETITIVAS

while

Este tipo de bucles se utilizan cuando queremos repetir la ejecución de unas sentencias un número indefinido de veces, siempre que se cumpla una condición. Este no es un proceso repetitivo por contador, es un proceso repetitivo por condición, es decir, va a evaluar una condición, si la condición se cumple ejecuta el conjunto de instrucciones que tiene y vuelve a evaluar la condición.... Y así sucesivamente hasta que la condición no se cumpla, en ese caso finaliza. Un bucle while, primero evalúa y luego ejecuta, por lo que se va a ejecutar 0 ó n veces.

```
while (condición) {  
    instrucciones;  
}
```

Ejemplo:

```
let i=0; while (i <=10) {  
    // Instrucciones a ejecutar mientras se cumpla la condición.  
    i++;  
}
```

ESTRUCTURAS REPETITIVAS

do ... while

Igual que el proceso repetitivo while, visto anteriormente, es un proceso repetitivo por condición, pero en este caso primero se ejecuta y después se evalúa, si la condición se cumple vuelve a ejecutar las instrucciones, y así hasta que no se cumpla la condición.

Es prácticamente igual que el bucle while(), con la diferencia, de que sabemos seguro que el bucle por lo menos se ejecutará siempre una vez, por lo tanto lo utilizaremos cuando el proceso repetitivo se va a ejecutar 1 ó N veces.

```
do {  
    instrucciones;  
} while (condición);
```

Ejemplo:

```
let a = 1; do{  
    alert("El valor de a es: "+a); // Mostrará esta alerta 2 veces.  
    a++;  
}while (a<3);
```

ESTRUCTURAS DE SALTO

break

`break;`

continue

`continue;`

labeled

`nombreEtiqueta: bucle`

`·
·
·`

`break nombreEtiqueta;`



Las instrucciones de salto break, continue y labeled son recursos que deben utilizarse en muy contadas ocasiones. En muchas empresas está completamente prohibido su uso, salvo casos muy excepcionales



06

ERRORES TÍPICOS EN JAVASCRIPT

ERRORES TÍPICOS EN JAVASCRIPT

Referencia al código antes de que esté cargado

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
  <script src="error.js"></script>
</head>
<body>

</body>
</html>
```


ERRORES TÍPICOS EN JAVASCRIPT

Uso incorrecto de los operadores == y ===

```
const num1 = 7;  
const num2 = "7";  
  
console.log (num1 == num2); //true  
console.log (num1 === num2); //false
```


ERRORES TÍPICOS EN JAVASCRIPT

Undefined no es lo mismo que Null

Undefined significa que una variable ha sido declarada pero no tiene asignada ningún valor. Null es un valor asignado a una variable.

```
//undefined
let var1;
console.log (var1); //undefined
console.log (typeof var1); //undefined

//null
let var2 = null;
console.log (var2); //null
console.log (typeof var2); //object

//Comprobamos que undefined no es lo mismo que null
var1 === var2 //false
undefined === null //false
```

ERRORES TÍPICOS EN JAVASCRIPT

Uso de sumas y concatenaciones

```
let suma = 21 + 2;  
console.log (suma); //23  
  
let concatenacion = 21 + "2";  
console.log (concatenacion); //212
```